Simple Local Search Sets 33 New Records for Balanced Latin Rectangles

Zhengyu Li
Georgia Institute of Technology
brian.li@gatech.edu

Vijay Ganesh
Georgia Institute of Technology
vganesh45@gatech.edu

Abstract

A $k \times n$ Latin rectangle is *balanced* when every pair of symbols lies, summed over the rows, at the same total distance; the *imbalance* measures how far a rectangle is from this ideal. Minimum-imbalance Latin rectangles are used in the design of agronomic field experiments, and finding them has become a benchmark for constraint programming, SAT, and, most recently, LLM-generated streamliners combined with mixed-integer programming. We show that a plain simulated-annealing search with incremental cost updates and an adaptive penalty for column conflicts improves the best known imbalance on 33 of the 51 non-square instances with $8 \leq n \leq 13$ studied in the literature, and matches the previous value on the remaining 18. The improvements average 18.5% and reach 39.4% (12×13: $274.0 \to 166.0$). The search is also orders of magnitude faster. Over 5 seeds per instance, it beats each previous record after a median of 0.12 CPU seconds on one laptop core, and every run beats it within 143 seconds, whereas the previous records came from pipelines that spend at least four hours per instance. It matches the four rectangles for which a SAT approach reports solving times of 32 minutes to 5.5 hours in at most 0.07 seconds. All rectangles, an independent checker, and the search code are public. The results are upper bounds: optimality remains open for all improved instances.

1 Introduction

Latin squares and rectangles are classical tools of experimental design. In agronomic field trials, treatments laid out in a Latin rectangle can still be biased by spatial autocorrelation when some pairs of treatments are systematically planted closer together than others [van Es and van Es, 1993]. *Spatially balanced* Latin squares, in which every pair of symbols has the same total distance, remove this bias [Gomes and Sellmann, 2004, Le Bras et al., 2012]. Díaz et al. [2017] extended the notion to $k \times n$ rectangles. Balanced rectangles exist only in restricted cases, so they defined the *imbalance* of a rectangle and asked for rectangles of minimum imbalance. They found this a hard optimization problem for CSP, local-search and MIP methods, and proposed it as a benchmark.

Since then the benchmark has been attacked with SAT encodings that also certify optimality via DRAT proofs [Peruvemba Ramaswamy and Szeider, 2023], and with streamliners generated by large language models and evaluated with Gurobi [Voboril et al., 2025], which improved the best known values on most open instances. Balanced Latin rectangles also serve as a test problem for LLM agents that reformulate constraint models [Voboril and Szeider, 2026].

Contribution. We revisit the problem with a deliberately simple method: simulated annealing [Kirkpatrick et al., 1983] over rows that are permutations, a swap move evaluated in $O(n)$ time, and a penalty for repeated symbols in a column whose weight adapts during the search, in the spirit of strategic oscillation [Glover and Hao, 2011]. None of these ingredients is new. The

point of this note is the outcome: on the instances of Voboril et al. [2025], the method improves 33 of 51 best known values and is never worse. It is also fast: on a single laptop core it beats each previous record within a median of 0.12 seconds, against pipelines that run for hours per instance. We release every rectangle together with an independent checker, so each record can be confirmed in seconds without trusting our solver.

We see two consequences. First, the previous upper bounds on this benchmark were much further from optimal than they appeared, which matters to anyone using them to judge new methods. Second, methods that aim at good solutions to this problem, including LLM-based ones, should be compared against a well-engineered local search and not only against an exact solver run under a time limit.

2 Problem

A $k \times n$ Latin rectangle ($k \le n$) is a $k \times n$ array over the symbols $\{1, \dots, n\}$ in which no symbol repeats in a row or in a column. Write $c_i(u)$ for the column of symbol u in row i. Following Peruvemba Ramaswamy and Szeider [2023], the distance of two symbols and the imbalance of L are

$$d(u,v) = \sum_{i=1}^{k} |c_i(u) - c_i(v)|, \qquad I(L) = \sum_{1 \le u < v \le n} \left| d(u,v) - \tfrac{k(n+1)}{3} \right|.$$

The target $k(n+1)/3$ is the average of $d(u,v)$ over all pairs, since every row contributes $\sum_{u<v} |c_i(u) - c_i(v)| = (n+1)n(n-1)/6$. Because $3I(L)$ is an integer, values are written $x.0$, $x.3$, $x.6$ for x, $x + \frac{1}{3}$, $x + \frac{2}{3}$, as in the literature. The goal is a rectangle of minimum imbalance for given k and n.

3 Method

Search space and move. The state is k permutations of the symbols, one per row. The first row is fixed to the identity, which loses no generality because relabelling the symbols does not change the imbalance. A move picks a row $i \ge 2$ and two columns $j_1 \ne j_2$ and swaps the symbols $a = L[i][j_1]$ and $b = L[i][j_2]$. Rows stay permutations, but the swap may create a repeated symbol in a column.

Incremental evaluation. We keep the matrix D of all pair distances. A swap in row i moves only a and b, so only the $2(n-2)$ distances $d(a,x)$ and $d(b,x)$ with $x \notin \{a,b\}$ change, and $d(a,b)$ does not change. For each such x, with $p_x = c_i(x)$,

$$d'(a,x) = d(a,x) - |j_1 - p_x| + |j_2 - p_x|, \qquad d'(b,x) = d(b,x) - |j_2 - p_x| + |j_1 - p_x|,$$

so the change of $3I(L)$ is computed exactly in $O(n)$ integer operations, against $O(kn^2)$ for recomputing the objective from scratch.

Column conflicts. Let V be the number of excess symbol occurrences over all columns, kept by per-column counters and updated in $O(1)$ per move. The search minimizes $3I(L) + W \cdot V$ and only records rectangles with $V = 0$. The weight W starts at k; every 2^{16} moves it is multiplied by 1.02 if the current state has conflicts and by 0.99 otherwise, clamped to $[1, 4n^2]$. This matters most when k is close to n: there, almost every swap creates a conflict, so a search that effectively forbids conflicts stalls. In a first version with a large fixed penalty ($W = 4n^2$), 10-second runs on 6×7 and

7×7 never visited a conflict-free rectangle other than the starting one. With the adaptive weight, 20-second runs reach the known optima of both (20.0 and 18.6).

Annealing schedule. The search starts from the cyclic rectangle $L[i][j] = i + j \bmod n$. A move with change Δ is accepted if $\Delta \leq 0$, and otherwise with probability $e^{-\Delta/T}$. The temperature starts at $T_0 = 2k$, is multiplied by 0.9999995 after every move, and is kept above 0.05; it is reset to T_0 when the best rectangle has not improved for $4 \cdot 10^7$ moves. Each run is single-threaded, stops after a fixed CPU time, and re-checks its best rectangle from scratch before reporting it. The implementation is about 130 lines of C.

4 Experiments

Setup. We use the instances of Table 1 of Voboril et al. [2025]: $8 \leq n \leq 13$ and $2 \leq k \leq n-1$, giving 51 cells. We omit squares ($k = n$), where the problem is governed by different structure [Xia et al., 2026]. The baseline in each cell is the best value reported there, which already includes earlier results and four-hour Gurobi runs. All runs were done on an Apple M3 laptop (8 cores, 16 GB), eight single-threaded runs at a time. We ran one run per cell with a budget of 1 minute, one with 5 minutes (different seed), and one with 10 minutes for the larger cells ($k \geq 4$, $n \geq 9$). We report the best rectangle per cell, so at most 16 CPU-minutes were spent on any instance.

Verification. Every reported rectangle is checked by an independent Python program written from the definition above, which uses exact rational arithmetic and shares no code with the solver. To make sure it computes exactly the published quantity, we ran it on all eleven rectangles printed in [Peruvemba Ramaswamy and Szeider, 2023, Voboril et al., 2025], including the eight CP 2025 records of Figure 6 there (up to 12×13). It reproduces every reported imbalance exactly.

Results. Table 1 and Figure 3 show the outcome. The search improves 33 cells, matches 18, and is worse on 0. All improved cells have $n \geq 9$ and $k \geq 3$. The matched cells are $k \in \{2, 3\}$ and $n = 8$, several of which were already proven optimal [Peruvemba Ramaswamy and Szeider, 2023], together with 4×10, 5×10 and 6×12. Gains grow with the size of the instance (Figure 4), up to 39.4% for 12×13. The one-minute runs alone already beat the previous record in 33 cells; longer runs mainly widen the margin (Figure 5).

Speed. To measure speed directly, we ran the search on every improved instance with 5 further seeds, each stopping as soon as its imbalance falls below the CP 2025 record, with a cap of five CPU minutes (Figure 1). All 165 runs beat the record. The median time is 0.12 seconds; the median per instance is below one second in 23 of 33 instances and below one minute in all of them (at most 54 seconds, for 9×10), and the slowest single run took 143 seconds. For comparison, each approach of Voboril et al. [2025] runs for 4 to 9 hours per instance, after a 4-hour Gurobi baseline, and they report that their incremental warmstart approach is still 25–60% worse than the baseline after one minute.

Peruvemba Ramaswamy and Szeider [2023] report the SAT solving time for four rectangles that improved the records of the time: 3×10 (imbalance 84, 32 minutes), 6×10 (96, 40 minutes), 4×11 (118, 102 minutes) and 6×11 (124, 5.5 hours). Our search reaches the same imbalance within 0.07 seconds in every run (Figure 2), at least 43,636 times faster by the median.

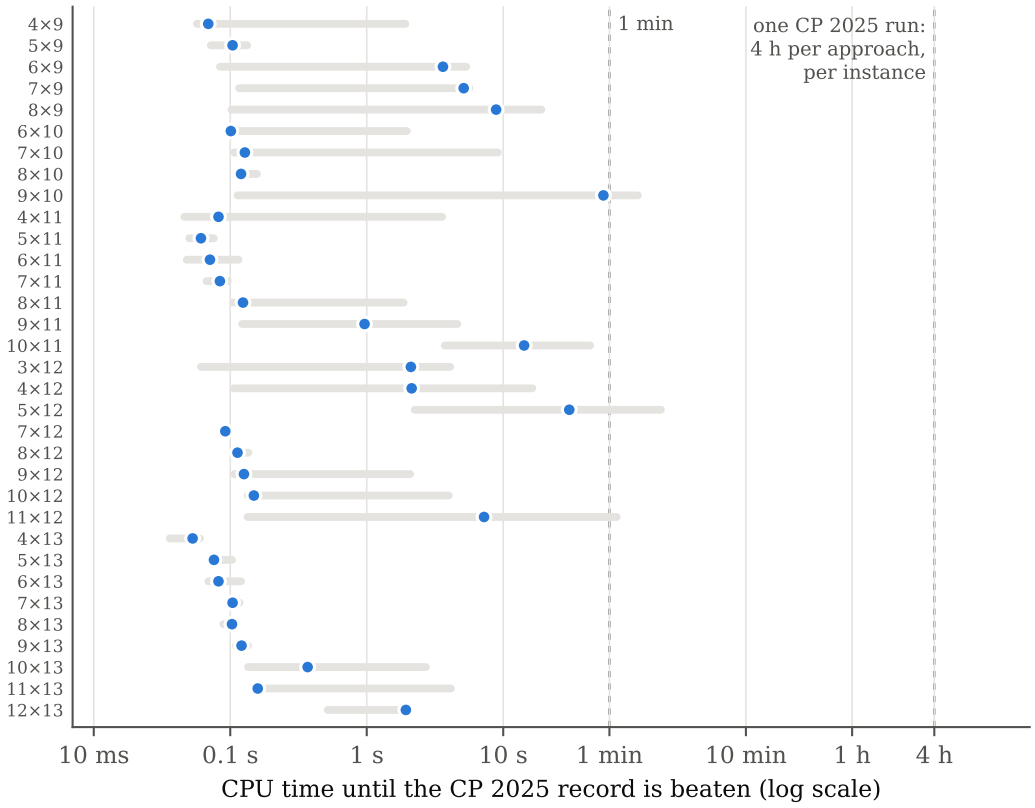


Figure 1: CPU time until the search beats the CP 2025 record, per instance: median (dot) and range (bar) over 5 seeds, one core, log scale. The dashed line at 4 hours marks the length of a single run of one CP 2025 approach on one instance.

5 Discussion

Why does a simple method do so well? We have not tested this, but the structure of the problem suggests an explanation. The objective is a sum of $\binom{n}{2}$ absolute deviations. A MIP model must linearize every one of them, and its LP relaxation can make all pairs look nearly balanced at once, so it likely gives the solver little guidance. Streamliners help the solver by restricting it to a structured subspace, but the best rectangles need not lie in that subspace. Local search has neither problem: the objective is cheap to update exactly, the swap neighbourhood is natural, and the penalty lets the search cross infeasible states. The setting suits annealing unusually well, and we do not claim that the comparison carries over to other problems.

How fair is the speed comparison? The gap spans four to five orders of magnitude, so we state its caveats plainly. First, the hardware differs: the earlier work used Intel Xeon E5-2640 v4 servers (2016), and we used an Apple M3 laptop (2023), whose single-core speed is higher by a small factor, not by orders of magnitude. Second, CP 2025 does not report when within its runs each record was found, so we compare our time to beat a record with the length of their runs, not with their time to reach it. Third, the CP 2023 times come from a complete SAT solver on a decision encoding; these instances are satisfiable, so the solver had to find a rectangle, the same task as ours, but its encoding is built to prove infeasibility as well. Finally, we did not rerun either

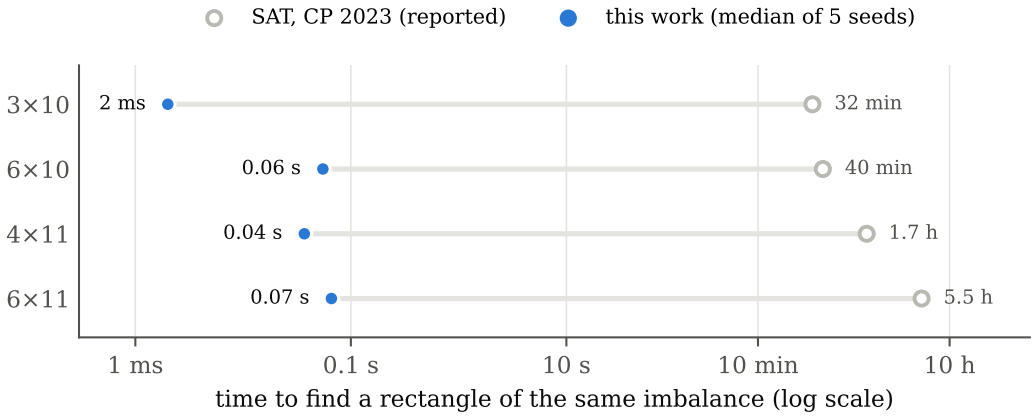


Figure 2: Time to find a rectangle with the imbalance that the SAT approach of Peruvemba Ramaswamy and Szeider [2023] reports, with its reported solving time. Ours: median of 5 seeds. Log scale.

$n\backslash k$	2	3	4	5	6	7	8	9	10	11	12
8	40.0	36.0	32.0	30.0	24.0	28.0					
9	65.3	56.0	**53.3** ~~56.0~~	**48.6** ~~52.6~~	**40.0** ~~48.0~~	**43.3** ~~51.3~~	**42.6** ~~53.3~~				
10	92.0	80.0	78.6	66.6	**60.0** ~~70.0~~	**58.0** ~~72.0~~	**58.6** ~~82.6~~	**64.0** ~~70.0~~			
11	124.0	114.0	**104.0** ~~112.0~~	**94.0** ~~116.0~~	**88.0** ~~108.0~~	**84.0** ~~118.0~~	**78.0** ~~114.0~~	**84.0** ~~120.0~~	**100.0** ~~110.0~~		
12	168.0	**152.0** ~~154.0~~	**149.3** ~~154.6~~	**140.0** ~~145.3~~	120.0	**121.3** ~~164.0~~	**116.6** ~~148.0~~	**106.0** ~~154.0~~	**124.0** ~~174.6~~	**136.6** ~~147.3~~	
13	218.6	198.0	**201.3** ~~211.3~~	**191.3** ~~210.6~~	**166.0** ~~204.0~~	**164.0** ~~203.3~~	**160.0** ~~220.6~~	**152.0** ~~214.0~~	**154.6** ~~240.6~~	**168.0** ~~224.0~~	**166.0** ~~274.0~~

Table 1: Best known imbalance of $k \times n$ Latin rectangles. Bold: new value from this work, with the previous best [Voboril et al., 2025] struck through below. Plain: value matched by our search.

pipeline ourselves. None of these caveats comes close to accounting for the gap.

Limitations. The new values are upper bounds. We prove nothing about optimality, and the size of the improvements suggests that the remaining gap to the optimum may still be large. The final records come from one run per cell and budget; only the speed experiment uses several seeds. Finally, the benchmark stops at $n = 13$; larger instances are an obvious next step.

6 Conclusion

A short, standard simulated-annealing search improves 33 of the 51 best-known imbalance values for balanced Latin rectangles studied in the recent literature, and matches the rest, beating each previous record within seconds on a laptop rather than hours on a server. Natural next steps are lower bounds or optimality proofs for the improved cells, for instance with the SAT approach of Peruvemba Ramaswamy and Szeider [2023], larger instances, and controlled comparisons with exact and LLM-guided methods on equal hardware.

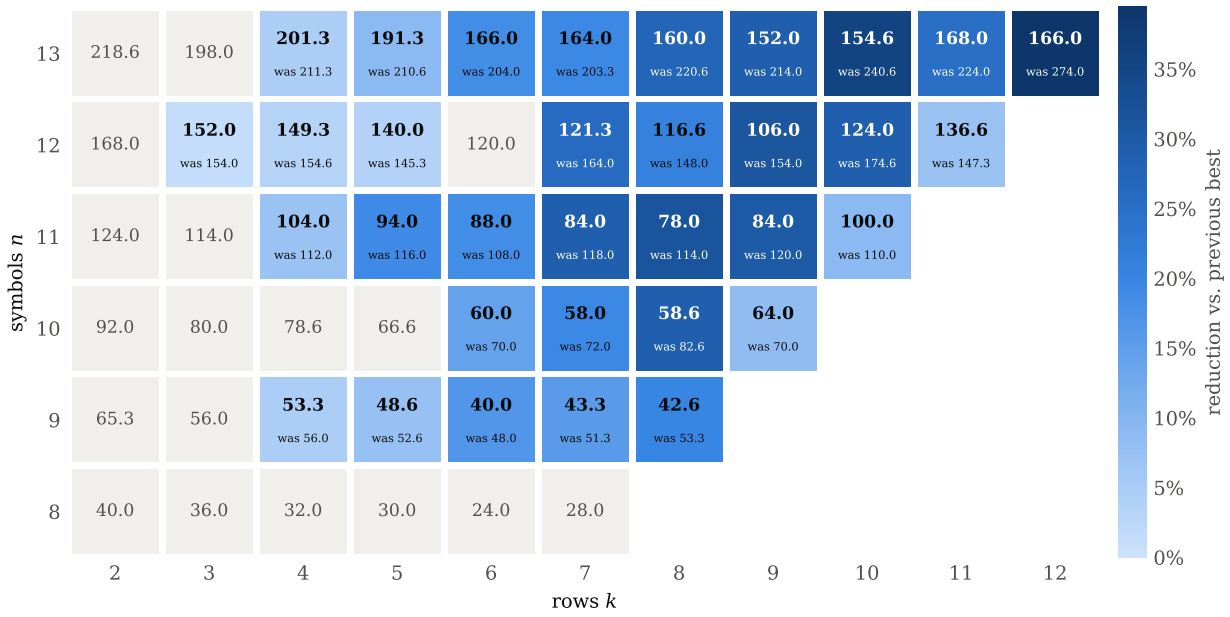


Figure 3: New best imbalance per cell (bold) with the previous best below it, shaded by the reduction relative to the previous best. Grey cells are matched, not improved.

Data and code. Rectangles, checker, solver and scripts: `https://github.com/BrianLi009/Balanced-Latin-Rectangles`.

References

Mateo Díaz, Ronan Le Bras, and Carla P. Gomes. In search of balance: The challenge of generating balanced Latin rectangles. In *Integration of AI and OR Techniques in Constraint Programming (CPAIOR 2017)*, volume 10335 of *Lecture Notes in Computer Science*, pages 68–76. Springer, 2017. doi: 10.1007/978-3-319-59776-8_6.

Fred Glover and Jin-Kao Hao. The case for strategic oscillation. *Annals of Operations Research*, 183(1):163–173, 2011. doi: 10.1007/s10479-009-0597-1.

Carla P. Gomes and Meinolf Sellmann. Streamlined constraint reasoning. In *Principles and Practice of Constraint Programming (CP 2004)*, volume 3258 of *Lecture Notes in Computer Science*, pages 274–289. Springer, 2004. doi: 10.1007/978-3-540-30201-8_22.

Scott Kirkpatrick, C. Daniel Gelatt, and Mario P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983. doi: 10.1126/science.220.4598.671.

Ronan Le Bras, Carla P. Gomes, and Bart Selman. From streamlined combinatorial search to efficient constructive procedures. In *Proceedings of the Twenty-Sixth AAAI Conference on Artificial Intelligence*, pages 499–506, 2012. doi: 10.1609/aaai.v26i1.8147.

Vaidyanathan Peruvemba Ramaswamy and Stefan Szeider. Proven optimally-balanced Latin rectangles with SAT (short paper). In *29th International Conference on Principles and Practice of Constraint Programming (CP 2023)*, volume 280 of *LIPIcs*, pages 48:1–48:10. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2023. doi: 10.4230/LIPIcs.CP.2023.48.

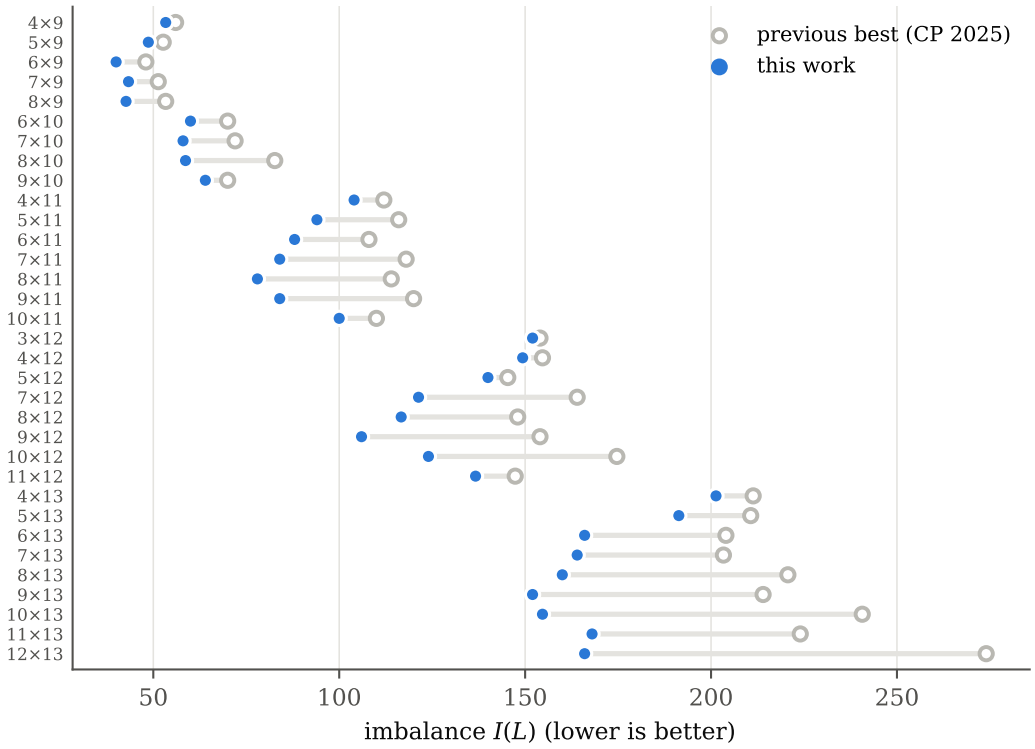


Figure 4: Previous best (open circles) and new value (filled) for every improved cell.

H. M. van Es and C. L. van Es. Spatial nature of randomization and its effect on the outcome of field experiments. *Agronomy Journal*, 85(2):420–428, 1993. doi: 10.2134/agronj1993.00021962008500020046x.

Florentina Voboril and Stefan Szeider. Improving constraint models with LLM agents, 2026. arXiv:2608.08127.

Florentina Voboril, Vaidyanathan Peruvemba Ramaswamy, and Stefan Szeider. Balancing Latin rectangles with LLM-generated streamliners. In *31st International Conference on Principles and Practice of Constraint Programming (CP 2025)*, volume 340 of *LIPIcs*, pages 36:1–36:17. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025. doi: 10.4230/LIPIcs.CP.2025.36.

Hai Xia, Carla P. Gomes, Bart Selman, and Stefan Szeider. Agentic neurosymbolic collaboration for mathematical discovery: A case study in combinatorial design, 2026. arXiv:2603.08322.

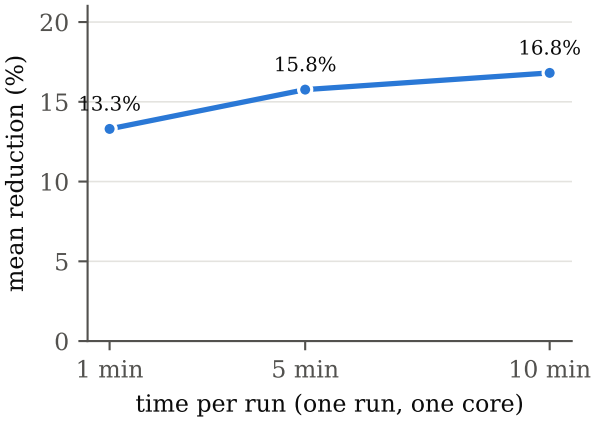


Figure 5: Mean reduction relative to the previous best over the cells with $k \geq 4$, $n \geq 9$, for a single run with each time budget. Each budget uses a different seed.